PATRÓN ESTRUCTURAL

Patrón de Diseño Estructural Decorator

Añadir responsabilidades a objetos
de forma dinámica.

Universidad Popular del Cesar · Ingeniería de Sistemas

Asignatura: Patrones de Diseño de Software
Integrantes: Juan Assia · Hernando Díaz
Juan Guzmán · David Palomino

Docente: Jairo Seoanes · Fecha: Octubre de 2026



DEFINICIÓN Y FUNDAMENTOS

Decorator es un patrón estructural que permite añadir responsabilidades a un objeto dinámicamente, mediante objetos decoradores (envoltorios) que conservan una interfaz compatible con la del objeto original.

01

CATEGORÍA

Patrón estructural

Organización de clases y objetos en estructuras más amplias.

02

PROPÓSITO

Extensión flexible

Extender el comportamiento de objetos en tiempo de ejecución.

03

MECANISMO

Composición

Delegación y envoltorios sucesivos (wrappers).

04

CONCEPTO CENTRAL

Composición flexible

Combina funcionalidades por composición, sin depender exclusivamente de la herencia para cada combinación.

EL PROBLEMA DE LA HERENCIA

¿Por qué necesitamos Decorator?

Ejemplo: sistema de notificaciones que envía correo, luego SMS y Facebook, y debe permitir combinar canales.

ENFOQUE RÍGIDO

Sin Decorator

Las subclases pueden multiplicarse al combinar comportamientos por herencia.

Combinaciones menos flexibles; diseño difícil de mantener y ampliar.

ENFOQUE MODULAR

Con Decorator

Cada responsabilidad es un decorador; se combinan por composición.

El cliente usa una interfaz común.

Aclaración: la multiplicación de clases depende del diseño adoptado y Decorator no elimina toda la complejidad.

¿Cuándo utilizar Decorator?

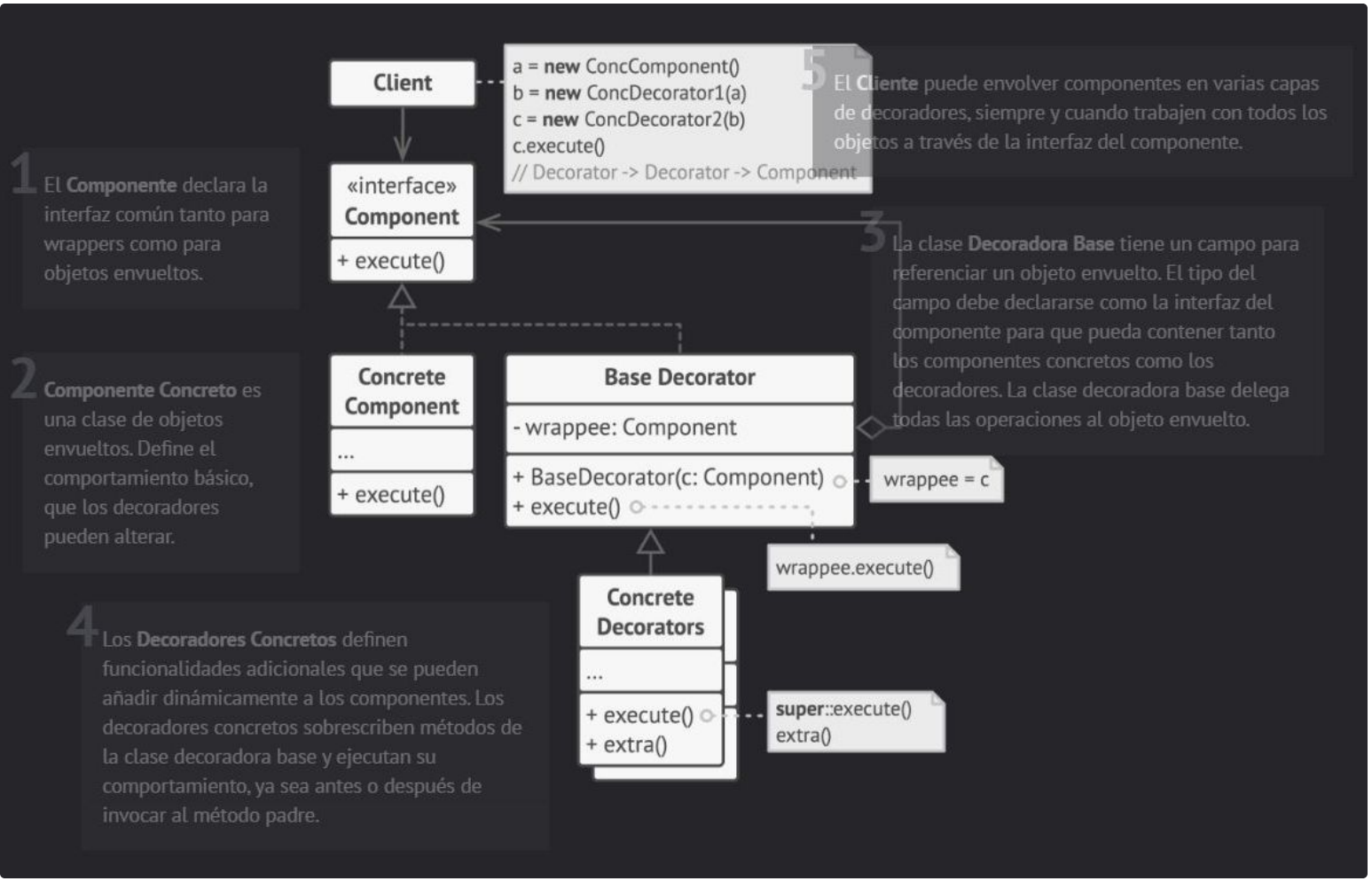
1. Funcionalidades opcionales para ciertos objetos
2. Comportamientos que se combinan de distintas maneras
3. Extender una clase sin modificar su implementación original
4. Responsabilidades agregadas en capas mediante envoltorios

Ejemplos: notificaciones, flujos de entrada y salida, compresión y codificación de datos, componentes de interfaces gráficas.

Advertencia: puede complicar la construcción y el seguimiento de objetos con muchas capas, o cuando el orden de los decoradores afecta el comportamiento.

ESTRUCTURA DEL PATRÓN

Arquitectura del patrón Decorator



COMPONENT

Interfaz común tanto para wrappers como para objetos envueltos.

CONCRETECOMPONENT

Objeto original con el comportamiento básico.

DECORATOR

Mantiene una referencia a un componente y delega las operaciones.

CONCRETEDECORATOR

Añade responsabilidades adicionales.

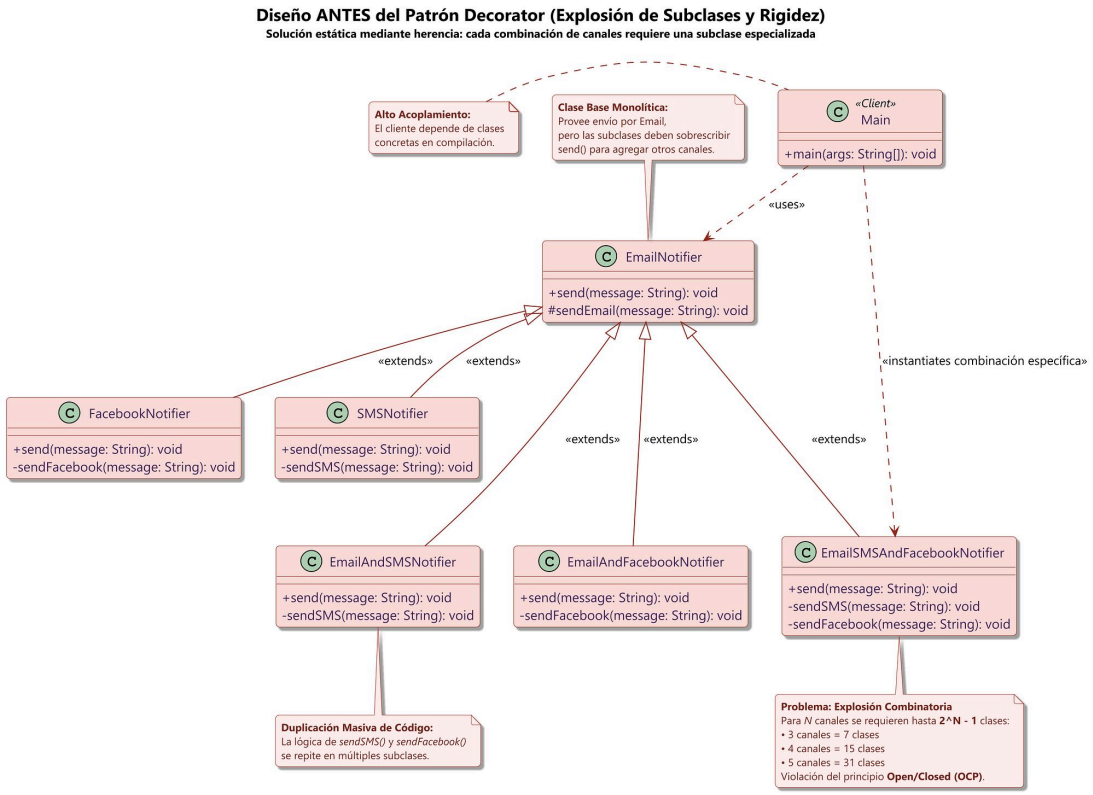
CLIENT

Usa los objetos a través de la interfaz común.

DISEÑO INICIAL SIN EL PATRÓN

Caso práctico: sistema de notificaciones sin Decorator

Cada combinación de canales requiere una subclase especializada; el diagrama y el código muestran la misma jerarquía.



Limitaciones

- **Rigidez:** cada combinación exige una subclase nueva.
- **Crecimiento:** las subclases se multiplican al combinar canales.
- Dificultad para extender el diseño.

Código Java sin Decorator (fragmento):

```java
class EmailNotifier {
  public void send(String message) {
    sendEmail(message);
  }
  protected void sendEmail(String message) {
    System.out.println("Email: " + message);
  }
}
class EmailAndSMSNotifier extends EmailNotifier {
  @Override public void send(String message) {
    sendEmail(message); sendSMS(message);
  }
  protected void sendSMS(String message) {
    System.out.println("SMS: " + message);
  }
}
// ... EmailAndFacebookNotifier, EmailSMSAndFacebookNotifier
new EmailAndSMSNotifier().send("Alerta");
```

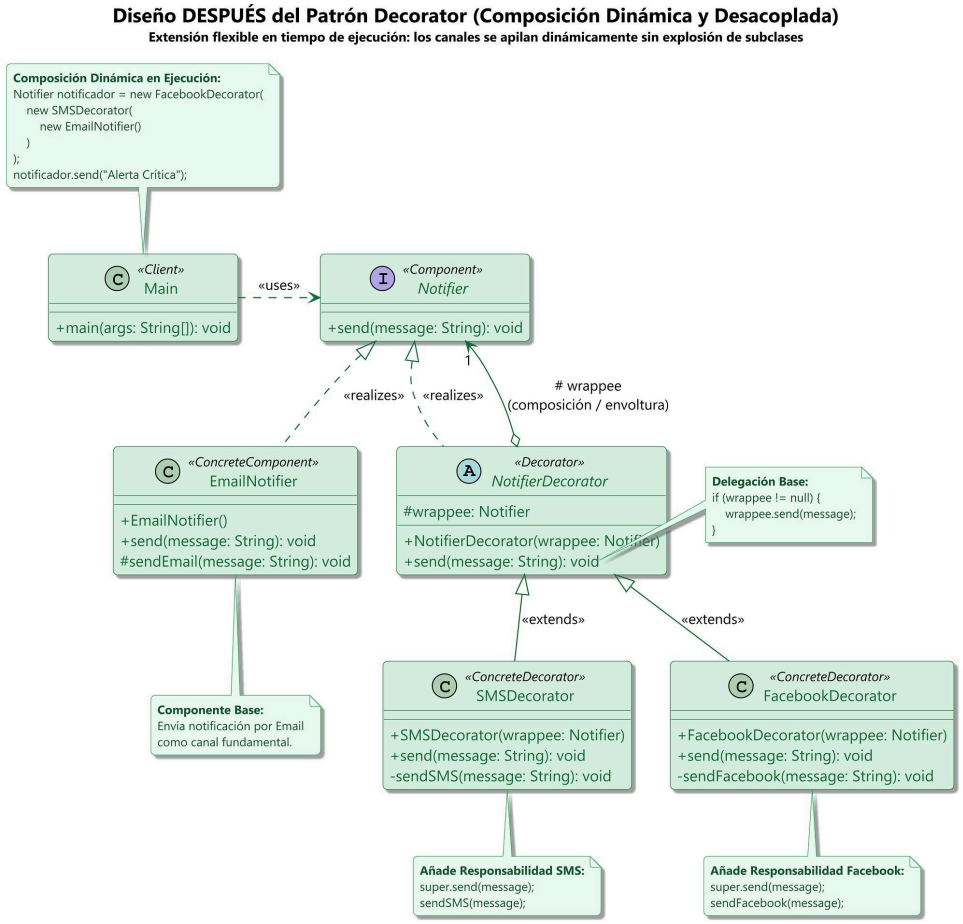
Fragmento ilustrativo: se omiten implementaciones y clases restantes.

Subclase para combinaciones: `EmailAndSMSNotifier`.

IMPLEMENTACIÓN CON EL PATRÓN

Aplicando Decorator al sistema

Los canales (SMS, Facebook) envuelven al notificador básico sin modificar EmailNotifier.



- **Composición:** cada decorador envuelve un Notificador.
- **Delegación:** send() reenvía la llamada al objeto envuelto.
- **Extensión:** cada decorador añade comportamiento sin cambiar las clases.

Código Java con Decorator (fragmento):

```java
interface Notificador { void send(String message); }
class EmailNotifier implements Notificador {
  @Override public void send(String message) {
    System.out.println("Email: " + message);
  }
}
abstract class NotifierDecorator implements Notificador {
  protected final Notificador wrappee;
  protected NotifierDecorator(Notificador wrappee) { this.wrappee = wrappee; }
  @Override public void send(String message) { wrappee.send(message); }
}
class SMSDecorator extends NotifierDecorator {
  SMSDecorator(Notificador w) { super(w); }
  @Override public void send(String message) {
    super.send(message);
    System.out.println("SMS: " + message);
  }
}
// FacebookDecorator: mismo patrón que SMSDecorator
Notificador n = new FacebookDecorator(new SMSDecorator(new EmailNotifier()));
n.send("Alerta");
```

Fragmento ilustrativo, no es una implementación completa.

Clases: `FacebookDecorator` envuelve a `SMSDecorator`, que envuelve a `EmailNotifier`.

ANÁLISIS COMPARATIVO

¿Qué cambia al aplicar Decorator?

Criterio	Antes (Sin Decorator)	Después (Con Decorator)
Extensibilidad	Rígida: exige modificar clases o heredar.	Flexible: se añaden decoradores sin alterar clases.
Combinaciones	Subclases fijas en tiempo de compilación (p. ej. EmailAndSMSNotifier).	Composición dinámica de envoltorios en tiempo de ejecución.
Acoplamiento	Alto: cliente atado a implementaciones especializadas.	Bajo: el cliente conoce solo la interfaz Notificador.
Código cliente	new EmailAndSMSNotifier().send(m)	new SMSDecorator(new EmailNotifier()).send(m)

Ventajas principales

Gran flexibilidad, cumplimiento del principio Open/Closed y separación de responsabilidades únicas por cada decorador.

Limitaciones y costos

Difícil retirar un decorador del centro de la pila; instanciación inicial con varios envoltorios.

RECAPITULACIÓN FINAL

Conclusiones

1. Decorator es un patrón estructural que usa composición y delegación para añadir responsabilidades de forma flexible

2. Problema que resuelve: extender objetos sin multiplicar subclases

3. Participantes: Component, ConcreteComponent, Decorator, ConcreteDecorator y Client

4. Evaluar cuándo conviene aplicarlo

Muchas gracias.

REPOSITORIO DEL PROYECTO

patron-decorator-java

URL:
https://github.com/David18Palomino/patron-decorator-java

Carpetas:

- before-decorator/
- after-decorator/

README con objetivo académico, instrucciones de ejecución, comparación técnica, pruebas automatizadas y diagramas UML de ambas implementaciones del patrón Decorator.